

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

Отчет
по лабораторной работе № 6
по курсу «Арифметические и логические основы вычислительной
техники»
на тему «Унарные и бинарные операции над графами»

Выполнили
студенты группы 24ВВВ2:

Макаров Я.С.

Бегичев Д.Д.

Родионов К.Е.

Приняли

Юрова О.В.

Деев М.В.

Пенза, 2025

Цель работы

Научиться проводить унарные и бинарные над графами.

Лабораторное задание

Задание 1

1 Сгенерируйте (используя генератор случайных чисел) две матрицы M_1, M_2

смежности неориентированных помеченных графов G_1, G_2 . Выведите сгенерированные матрицы на экран.

2 * Для указанных графов преобразуйте представление матриц смежности в списки смежности. Выведите полученные списки на экран.

Задание 2

1 Для матричной формы представления графов выполните операцию:

- а) отождествления вершин
- б) стягивания ребра
- в) расщепления вершины

Номера выбираемых для выполнения операции вершин ввести с клавиатуры.

Результат выполнения операции выведите на экран.

2 * Для представления графов в виде списков смежности выполните операцию:

- а) отождествления вершин
- б) стягивания ребра
- в) расщепления вершины

Номера выбираемых для выполнения операции вершин ввести с клавиатуры.

Результат выполнения операции выведите на экран.

Задание 3

1 Для матричной формы представления графов выполните операцию:

- а) объединения $G = G_1 \cup G_2$
- б) пересечения $G = G_1 \cap G_2$

в) кольцевой суммы $G = G1 \oplus G2$

Результат выполнения операции выведите на экран.

Задание 4 *

1 Для матричной формы представления графов выполните операцию декартова произведения графов $G = G1 \times G2$.

Результат выполнения операции выведите на экран.

Описание метода решения

Для выполнения работы по унарным операциям над графами используется систематический подход, основанный на последовательном применении операций из обеих групп. Работа выполняется путем анализа исходного графа G_1 и применения к нему унарных операций согласно поставленным задачам. Каждая операция изменяет структуру графа, создавая новый граф G_2 с измененными характеристиками. К операциям первой группы, уменьшающим число элементов, относятся отождествление вершин и стягивание ребра. При отождествлении вершин две вершины объединяются в одну, при этом все ребра, инцидентные исходным вершинам, становятся инцидентными новой вершине. Стягивание ребра предполагает удаление ребра и отождествление его концевых вершин в одну вершину. Вторая группа операций позволяет увеличивать число элементов графа и включает расщепление вершины и добавление ребра. При расщеплении вершина разделяется на две или более вершин, а инцидентные ребра распределяются между новыми вершинами. Добавление ребра означает введение в граф нового ребра, соединяющего две существующие вершины. Порядок выполнения работы следующий: сначала исходный граф представляется в виде матрицы смежности или списка ребер, затем определяется требуемая операция из задания, после чего операция применяется с модификацией структуры данных, результат визуализируется и проверяется корректность полученного графа. Этот метод позволяет систематически преобразовывать графы и анализировать изменения их свойств.

Псевдокод

1-ое задание

алг генерация_двух_графов

дано

количество вершин peak1 для графа G1

количество вершин peak2 для графа G2

надо

создать случайные неориентированные графы G1 и G2 без петель

вывести матрицу смежности и список смежности для каждого графа

освободить память

нач цел

peak1, peak2 := количество вершин

M1, M2 := матрицы смежности

// Процедура создания матрицы смежности

процедура createMatrix(matrix, peak)

нач

нц для i от 0 до peak-1

нц для j от i+1 до peak-1

value := случайное число 0 или 1

matrix[i][j] := value

matrix[j][i] := value

кц

matrix[i][i] := 0

кц

кон

// Процедура вывода матрицы смежности

процедура printMatrix(matrix, peak)

нач

вывести "Матрица смежности:"

нц для i от 0 до peak-1

нц для j от 0 до peak-1

вывести matrix[i][j]

кц

перевод строки

кц

кон

// Процедура вывода списка смежности

процедура makeList(matrix, peak)

нач

вывести "Список смежности:"

нц для i от 0 до peak-1

вывести "Вершина", $i+1$, ":"

peak := 0

нц для j от 0 до peak-1

если matrix[i][j] = 1 и $i \neq j$ тогда

вывести $j+1$

peak := 1

конец если

кц

если peak = 0 тогда вывести "Нет соседей"

перевод строки

кц

кон

// Процедура освобождения памяти

процедура freeM(matrix, peak)

нач

нц для i от 0 до peak-1

освободить matrix[i]

кц

освободить matrix

кон

// Основная программа

нач

установить локаль

инициализировать генератор случайных чисел

вывести "Введите количество вершин для G1:"

ввести peak1

вывести "Введите количество вершин для G2:"

ввести peak2

если $\text{peak1} \leq 0$ или $\text{peak2} \leq 0$ тогда

вывести "Ошибка: количество вершин должно быть натуральным"

завершение

выделить память под M1 размером $\text{peak1} \times \text{peak1}$

выделить память под M2 размером $\text{peak2} \times \text{peak2}$

вывести "Граф G1"

вызов createMatrix(M1, peak1)

вызов printMatrix(M1, peak1)

вызов makeList(M1, peak1)

вывести "Граф G2"

вызов createMatrix(M2, peak2)

вызов printMatrix(M2, peak2)

вызов makeList(M2, peak2)

вызов freem(M1, peak1)

вызов freem(M2, peak2)

кОН

2-ое задание

алг операции_над_графами

дано

количество вершин peak

выбор представления графа (матрица или списки смежности)

операции: отождествление вершин, стягивание ребра, расщепление вершины

надо

создать случайный граф

выполнять операции над графом согласно выбору пользователя

выводить результаты

нач цел

представление, peak, выбор, p1, p2, p, newpeak

// Процедура создания матрицы смежности

процедура createMatrixAdj(matrix, peak)

нач

нц для i от 0 до peak-1

нц для j от i+1 до peak-1

value := случайное 0 или 1

matrix[i][j] := value

matrix[j][i] := value

кц

matrix[i][i] := 0

кц

кОН

// Процедура вывода матрицы смежности

процедура printMatrixAdj(matrix, peak)

нач

вывести заголовок с номерами вершин

нц для i от 0 до peak-1

вывести номер вершины i+1

нц для j от 0 до peak-1

вывести matrix[i][j]

кц

перевод строки

кц

кон

// Функция копирования матрицы

функция copyMatrixAdj(source, peak)

нач

выделить память под buffer размером $\text{peak} \times \text{peak}$

нц для i от 0 до peak-1

нц для j от 0 до peak-1

buffer[i][j] := source[i][j]

кц

кц

вернуть buffer

кон

// Функция отождествления вершин (матрица)

функция identifiypeakMatrix(matrix, peak, p1, p2, newpeak)

нач

если p1, p2 некорректны тогда

вывести "Ошибка"

newpeak := peak

```

вернуть copyMatrixAdj(matrix, peak)
i1 := p1-1, i2 := p2-1
keepi := min(i1, i2), deli := max(i1, i2)
newpeak := peak - 1
выделить память под buffer размером newpeak × newpeak
нц для i от 0 до peak-1
если i = deli пропустить
new_i := если i < deli то i иначе i-1
нц для j от 0 до peak-1
если j = deli пропустить
new_j := если j < deli то j иначе j-1
если i = keepi или j = keepi тогда
buffer[new_i][new_j] := объединение связей keepi и deli
иначе
buffer[new_i][new_j] := matrix[i][j]
кц
кц
вернуть buffer
кон

```

// Функция стягивания ребра (матрица)

```

функция mergeMatrix(matrix, peak, p1, p2, newpeak)
нач
если p1, p2 некорректны тогда вернуть копию
i1 := p1-1, i2 := p2-1
если matrix[i1][i2] = 0 тогда
вывести "Ошибка: ребро не существует"
вернуть копию
вернуть identifypeakMatrix(matrix, peak, p1, p2, newpeak)
кон

```

```

// Функция расщепления вершины (матрица)
функция splitMatrix(matrix, peak, p, newpeak)
нач
если p некорректен вернуть копию
i := p-1
newpeak := peak + 1
выделить память под buffer размером newpeak × newpeak
скопировать matrix в buffer
newpi := peak
buffer[i][newpi] := 1
buffer[newpi][i] := 1
обнулить диагональ
вернуть buffer
кон

```

```

// Процедура освобождения памяти матрицы
процедура freemMatrix(matrix, peak)
нач
нц для i от 0 до peak-1
освободить matrix[i]
кц
освободить matrix
кон

```

```

// Структуры для списков смежности
структура Node (вершина, указатель next)
структура Graph (peak, массив списков adjLists)

```

```

// Функция создания узла
функция createNode(v)
нач
выделить память под newNode

```

newNode.vertex := v

newNode.next := NULL

вернуть newNode

кон

// Функция создания графа

функция createGraph(peak)

нач

выделить память под graph

graph.peak := peak

выделить массив списков смежности

инициализировать все списки как NULL

вернуть graph

кон

// Процедура добавления ребра

процедура addEdge(graph, src, dest)

нач

создать узел для dest и добавить в список src

создать узел для src и добавить в список dest

кон

// Процедура создания случайного графа (списки)

процедура createMatrixList(graph, peak)

нач

нц для i от 0 до peak-1

нц для j от i+1 до peak-1

если случайное = 1 тогда addEdge(graph, i, j)

кц

кц

кон

// Процедура вывода списков смежности

процедура printMatrixList(graph)

нач

нц для i от 0 до graph.peak-1

вывести "Вершина", i+1, ":"

temp := graph.adjLists[i]

если temp = NULL вывести "нет соседей"

иначе

нц пока temp ≠ NULL

вывести temp.vertex + 1

temp := temp.next

кц

перевод строки

кц

кон

// Процедура удаления вершины из списка

процедура removeVertexFromList(head, v)

нач

temp := head, prev := NULL

нц пока temp ≠ NULL и temp.vertex ≠ v

prev := temp, temp := temp.next

кц

если temp ≠ NULL тогда

если prev = NULL head := temp.next

иначе prev.next := temp.next

освободить temp

кон

// Функция проверки наличия вершины в списке

функция isVertexInList(head, v)

нач

temp := head

нц пока temp $\neq$ NULL

если temp.vertex = v вернуть 1

temp := temp.next

кц

вернуть 0

кон

// Функция отождествления вершин (списки)

функция identifypeakList(graph, p1, p2, newpeak)

нач

если p1, p2 некорректны вернуть копию

i1 := p1-1, i2 := p2-1

keepi := min(i1, i2), deli := max(i1, i2)

newpeak := graph.peak - 1

создать newGraph размером newpeak

объединить списки смежности keepi и deli

скопировать остальные списки с корректировкой индексов

вернуть newGraph

кон

// Функция стягивания ребра (списки)

функция mergeList(graph, p1, p2, newpeak)

нач

если p1, p2 некорректны вернуть копию

i1 := p1-1, i2 := p2-1

если ребро не существует вернуть копию

result := identifypeakList(graph, p1, p2, newpeak)

удалить петлю из объединённой вершины

вернуть result

кон

// Функция расщепления вершины (списки)

функция splitList(graph, p, newpeak)

нач

если p некорректен вернуть копию

$i := p-1$

newpeak := graph.peak + 1

создать newGraph размером newpeak

скопировать все списки

newpi := graph.peak

добавить ребро между i и newpi

вернуть newGraph

кон

// Процедура освобождения графа

процедура freeList(graph)

нач

нц для i от 0 до graph.peak-1

освободить все узлы в списке i

кц

освободить graph

кон

// Основная программа

нач

установить локаль

инициализировать генератор случайных чисел

вывести "Выберите представление: 1-матрица, 2-списки"

ввести представление

если представление $\neq 1$ и $\neq 2$ тогда

вывести "Ошибка"

завершение

вывести "Введите количество вершин:"

ввести peak

если $\text{peak} \leq 0$ тогда

вывести "Ошибка"

завершение

если представление = 1 тогда

выделить память под matrix размером $\text{peak} \times \text{peak}$

вызов createMatrixAdj(matrix, peak)

вывести исходный граф

вызов printMatrixAdj(matrix, peak)

нц пока выбор $\neq 0$

вывести меню операций

ввести выбор

выбор выбор из

случай 1: ввести p1, p2, выполнить отождествление, вывести результат

случай 2: ввести p1, p2, выполнить стягивание, вывести результат

случай 3: ввести p, выполнить расщепление, вывести результат

случай 0: выход

конец выбора

кц

вызов freeMatrix(matrix, peak)

иначе если представление = 2 тогда

graph := createGraph(peak)

вызов createMatrixList(graph, peak)

вывести исходный граф

вызов printMatrixList(graph)

нц пока выбор $\neq 0$

вывести меню операций

ввести выбор

выбор выбор из

случай 1: ввести p1, p2, выполнить отождествление, вывести результат

случай 2: ввести p1, p2, выполнить стягивание, вывести результат

случай 3: ввести p, выполнить расщепление, вывести результат

случай 0: выход

конец выбора

кц

вызов freeMList(graph)

кон

3-ие задание

алг операции_над_двумя_графами

дано

количество вершин peak для графов G1 и G2

матрицы смежности matrix1 и matrix2

надо

создать два случайных графа

выполнять операции: объединение, пересечение, кольцевая сумма

выводить результаты операций

нач цел

peak, выбор

matrix1, matrix2, result

// Процедура создания случайной матрицы смежности

процедура createMatrix(matrix, peak)

нач

нц для i от 0 до peak-1

нц для j от i+1 до peak-1

value := случайное 0 или 1

matrix[i][j] := value

matrix[j][i] := value

кц

matrix[i][i] := 0

кц

кон

// Процедура вывода матрицы смежности

процедура printMatrix(matrix, peak, name)

нач

вывести name, размер peak × peak

вывести заголовок с номерами вершин

нц для i от 0 до peak-1

вывести номер вершины i+1

нц для j от 0 до peak-1

вывести matrix[i][j]

кц

перевод строки

кц

перевод строки

кон

// Функция создания матрицы с выделением памяти

функция createMatrixFunc(peak)

нач

выделить память под matrix размером $\text{peak} \times \text{peak}$
инициализировать нулями
вернуть matrix
конец

// Процедура освобождения памяти матрицы

процедура freem(matrix, peak)

нач

нц для i от 0 до peak-1

освободить matrix[i]

кц

освободить matrix

конец

// Функция копирования матрицы

функция copyMatrix(source, peak)

нач

copy := createMatrixFunc(peak)

нц для i от 0 до peak-1

нц для j от 0 до peak-1

copy[i][j] := source[i][j]

кц

кц

вернуть copy

конец

// Функция объединения графов (логическое ИЛИ)

функция unionGraphs(matrix1, matrix2, peak)

нач

result := createMatrixFunc(peak)

нц для i от 0 до peak-1

нц для j от 0 до peak-1

```
result[i][j] := если matrix1[i][j] или matrix2[i][j] то 1 иначе 0
```

```
кц
```

```
кц
```

```
вернуть result
```

```
кон
```

```
// Функция пересечения графов (логическое И)
```

```
функция intersectionGraphs(matrix1, matrix2, peak)
```

```
нач
```

```
result := createMatrixFunc(peak)
```

```
нц для i от 0 до peak-1
```

```
нц для j от 0 до peak-1
```

```
result[i][j] := если matrix1[i][j] и matrix2[i][j] то 1 иначе 0
```

```
кц
```

```
кц
```

```
вернуть result
```

```
кон
```

```
// Функция кольцевой суммы графов (XOR)
```

```
функция ringSumGraphs(matrix1, matrix2, peak)
```

```
нач
```

```
result := createMatrixFunc(peak)
```

```
нц для i от 0 до peak-1
```

```
нц для j от 0 до peak-1
```

```
result[i][j] := если matrix1[i][j]  $\neq$  matrix2[i][j] то 1 иначе 0
```

```
кц
```

```
кц
```

```
вернуть result
```

```
кон
```

```
// Основная программа
```

```
нач
```

установить локаль

инициализировать генератор случайных чисел

вывести "Введите количество вершин для G1 и G2:"

ввести peak

если $\text{peak} \leq 0$ тогда

вывести "Ошибка: количество вершин должно быть положительным"

завершение

$\text{matrix1} := \text{createMatrixFunc}(\text{peak})$

$\text{matrix2} := \text{createMatrixFunc}(\text{peak})$

вызов $\text{createMatrix}(\text{matrix1}, \text{peak})$

вызов $\text{createMatrix}(\text{matrix2}, \text{peak})$

вывести "Исходные графы"

вызов $\text{printMatrix}(\text{matrix1}, \text{peak}, \text{"Матрица смежности G1"})$

вызов $\text{printMatrix}(\text{matrix2}, \text{peak}, \text{"Матрица смежности G2"})$

нц пока $\text{выбор} \neq 0$

вывести меню операций:

"1 - Объединение $G1 \cup G2$ "

"2 - Пересечение $G1 \cap G2$ "

"3 - Кольцевая сумма $G1 \oplus G2$ "

"0 - Выход"

ввести выбор

$\text{result} := \text{NULL}$

$\text{operationName} := ""$

выбор выбор из

случай 1:

$\text{result} := \text{unionGraphs}(\text{matrix1}, \text{matrix2}, \text{peak})$

$\text{operationName} := \text{"Результат объединения G1 U G2"}$

случай 2:

result := intersectionGraphs(matrix1, matrix2, peak)

operationName := "Результат пересечения $G1 \cap G2$ "

случай 3:

result := ringSumGraphs(matrix1, matrix2, peak)

operationName := "Результат кольцевой суммы $G1 \oplus G2$ "

случай 0:

вывести "Выход из программы"

иначе:

вывести "Неверный выбор"

конец выбора

если result $\neq$ NULL тогда

вывести operationName

вызов printMatrix(result, peak, "")

вызов freem(result, peak)

кц

вызов freem(matrix1, peak)

вызов freem(matrix2, peak)

кон

4-ое задание

алг декартово_произведение_графов

дано

количество вершин peak1 для графа G1

количество вершин peak2 для графа G2

матрицы смежности matrix1 и matrix2

надо

создать два случайных графа $G1$ и $G2$

вычислить декартово произведение $G1 \times G2$

вывести матрицу смежности и списки смежности результата

нач цел

peak1, peak2, resultPeak

matrix1, matrix2, result

// Процедура создания случайной матрицы смежности

процедура createMatrix(matrix, peak)

нач

нц для i от 0 до peak-1

нц для j от $i+1$ до peak-1

value := случайное 0 или 1

matrix[i][j] := value

matrix[j][i] := value

кц

matrix[i][i] := 0

кц

кон

// Процедура вывода матрицы смежности

процедура printMatrix(matrix, peak, name)

нач

вывести name, размер peak $\times$ peak

вывести заголовок с номерами вершин от 1 до peak

нц для i от 0 до peak-1

вывести номер вершины $i+1$

нц для j от 0 до peak-1

вывести matrix[i][j]

кц

перевод строки

кц

перевод строки

кон

// Функция создания матрицы с выделением памяти

функция createMatrixFunc(peak)

нач

выделить память под matrix размером $\text{peak} \times \text{peak}$

инициализировать нулями

вернуть matrix

кон

// Процедура освобождения памяти

процедура freeM(matrix, peak)

нач

нц для i от 0 до peak-1

освободить matrix[i]

кц

освободить matrix

кон

// Функция вычисления декартова произведения

функция cartesianProduct(matrix1, peak1, matrix2, peak2, resultPeak)

нач

resultPeak := peak1 * peak2

result := createMatrixFunc(resultPeak)

нц для u1 от 0 до peak1-1

нц для v1 от 0 до peak2-1

нц для u2 от 0 до peak1-1

нц для v2 от 0 до peak2-1

i := u1 * peak2 + v1

$j := u_2 * peak_2 + v_2$

если $(u_1 = u_2 \text{ и } matrix_2[v_1][v_2] = 1)$ или $(v_1 = v_2 \text{ и } matrix_1[u_1][u_2] = 1)$ тогда
 $result[i][j] := 1$

конец если

кц

кц

кц

кц

вернуть result

кон

// Процедура вывода матрицы декартова произведения с парными индексами
процедура printCartesianMatrix(matrix, peak1, peak2, name)

нач

$totalPeak := peak_1 * peak_2$

вывести name, размер $totalPeak \times totalPeak$

вывести заголовок с парами вершин (u, v)

нц для u_1 от 0 до $peak_1 - 1$

нц для v_1 от 0 до $peak_2 - 1$

$i := u_1 * peak_2 + v_1$

вывести пару $(u_1 + 1, v_1 + 1)$

нц для u_2 от 0 до $peak_1 - 1$

нц для v_2 от 0 до $peak_2 - 1$

$j := u_2 * peak_2 + v_2$

вывести $matrix[i][j]$

кц

кц

перевод строки

кц

кц

перевод строки

кон

// Процедура вывода списков смежности декартова произведения

процедура printCartesianAdjacencyList(matrix, peak1, peak2)

нач

totalPeak := peak1 * peak2

вывести "Списки смежности декартова произведения:"

нц для u1 от 0 до peak1-1

нц для v1 от 0 до peak2-1

i := u1 * peak2 + v1

вывести "Вершина (" , u1+1, ",", v1+1, "): "

hasNeighbors := 0

нц для u2 от 0 до peak1-1

нц для v2 от 0 до peak2-1

j := u2 * peak2 + v2

если matrix[i][j] = 1 и i ≠ j тогда

вывести "(" , u2+1, ",", v2+1, ") "

hasNeighbors := 1

конец если

кц

кц

если hasNeighbors = 0 тогда вывести "нет соседей"

перевод строки

кц

кц

перевод строки

кон

// Основная программа

нач

установить локаль

инициализировать генератор случайных чисел

вывести "Введите количество вершин для G1:"

ввести peak1

вывести "Введите количество вершин для G2:"

ввести peak2

если $\text{peak1} \leq 0$ или $\text{peak2} \leq 0$ тогда

вывести "Ошибка: количество вершин должно быть положительным"

завершение

`matrix1 := createMatrixFunc(peak1)`

`matrix2 := createMatrixFunc(peak2)`

вызов `createMatrix(matrix1, peak1)`

вызов `createMatrix(matrix2, peak2)`

вывести "Исходные графы"

вызов `printMatrix(matrix1, peak1, "Матрица смежности G1")`

вызов `printMatrix(matrix2, peak2, "Матрица смежности G2")`

`result := cartesianProduct(matrix1, peak1, matrix2, peak2, resultPeak)`

вывести "Декартово произведение $G1 \times G2$ "

вывести "Количество вершин:", peak1, "×", peak2, "=", resultPeak

вызов `printCartesianMatrix(result, peak1, peak2, "Матрица смежности декартова произведения")`

вызов `printCartesianAdjacencyList(result, peak1, peak2)`

вызов `freem(matrix1, peak1)`

вызов `freem(matrix2, peak2)`

вызов `freem(result, resultPeak)`

КОН

Листинг

1-ое задание

```
#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h>

#include <stdlib.h>

#include <time.h>

#include <locale.h>

void createMatrix(int** matrix, int peak) {
    for (int i = 0; i < peak; i++) {
        for (int j = i + 1; j < peak; j++) {
            int value = rand() % 2;
            matrix[i][j] = value;
            matrix[j][i] = value;
        }
        matrix[i][i] = 0;
    }
}

void printMatrix(int** matrix, int peak) {
    printf("Матрица смежности:\n");
    for (int i = 0; i < peak; i++) {
        for (int j = 0; j < peak; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
    printf("\n");
}

void makeList(int** matrix, int peak) {
    printf("Список смежности:\n");
    for (int i = 0; i < peak; i++) {
```

```

        printf("Вершина %d: ", i + 1);
        int near = 0;
        for (int j = 0; j < peak; j++) {
            if (matrix[i][j] == 1 && i != j) {
                printf("%d ", j + 1);
                near = 1;
            }
        }
        if (!near) {
            printf("Нет соседей");
        }
        printf("\n");
    }
    printf("\n");
}

void freem(int** matrix, int peak) {
    for (int i = 0; i < peak; i++) {
        free(matrix[i]);
    }
    free(matrix);
}

int main() {
    setlocale(LC_ALL, "ru");
    srand(time(NULL));
    int peak1, peak2;

    printf("Введите количество вершин для графа G1: ");
    scanf("%d", &peak1);
    printf("Введите количество вершин для графа G2: ");
    scanf("%d", &peak2);

```

```

    if (peak1 <= 0 || peak2 <= 0) {
        printf("Ошибка: количество вершин должно быть
натуральным числом.\n");
        return 1;
    }

    int** M1 = (int**)malloc(peak1 * sizeof(int*));
    for (int i = 0; i < peak1; i++) {
        M1[i] = (int*)malloc(peak1 * sizeof(int));
    }

    int** M2 = (int**)malloc(peak2 * sizeof(int*));
    for (int i = 0; i < peak2; i++) {
        M2[i] = (int*)malloc(peak2 * sizeof(int));
    }

    printf("\nГраф G1\n");
    createMatrix(M1, peak1);
    printMatrix(M1, peak1);
    makeList(M1, peak1);

    printf("Граф G2\n");
    createMatrix(M2, peak2);
    printMatrix(M2, peak2);
    makeList(M2, peak2);

    freem(M1, peak1);
    freem(M2, peak2);
    return 0;
}

```

2-ое задание

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>

typedef struct Node {
    int vertex;
    struct Node* next;
} Node;

typedef struct {
    int peak;
    Node** adjLists;
} Graph;

void createMatrixAdj(int** matrix, int peak) {
    for (int i = 0; i < peak; i++) {
        for (int j = i + 1; j < peak; j++) {
            int value = rand() % 2;
            matrix[i][j] = value;
            matrix[j][i] = value;
        }
        matrix[i][i] = 0;
    }
}

void printMatrixAdj(int** matrix, int peak) {
    printf("Матрица смежности (%dx%d):\n", peak, peak);
    printf("    ");
    for (int j = 0; j < peak; j++) {
```

```

        printf("%2d ", j + 1);
    }
    printf("\n");

    for (int i = 0; i < peak; i++) {
        printf("%2d ", i + 1);
        for (int j = 0; j < peak; j++) {
            printf("%2d ", matrix[i][j]);
        }
        printf("\n");
    }
    printf("\n");
}

int** copyMatrixAdj(int** source, int peak) {
    int** buffer = (int**)malloc(peak * sizeof(int*));
    for (int i = 0; i < peak; i++) {
        buffer[i] = (int*)malloc(peak * sizeof(int));
        for (int j = 0; j < peak; j++) {
            buffer[i][j] = source[i][j];
        }
    }
    return buffer;
}

int** identifiypeakMatrix(int** matrix, int peak, int p1, int p2,
int* newpeak) {
    if (p1 < 1 || p1 > peak || p2 < 1 || p2 > peak || p1 == p2)
    {
        printf("Ошибка: некорректные номера вершин!\n");
        *newpeak = peak;
        return copyMatrixAdj(matrix, peak);
    }

```



```

int i1 = p1 - 1;
int i2 = p2 - 1;
int keepi = (i1 < i2) ? i1 : i2;
int deli = (i1 < i2) ? i2 : i1;

*newpeak = peak - 1;

int** buffer = (int**)malloc(*newpeak * sizeof(int*));
for (int i = 0; i < *newpeak; i++) {
    buffer[i] = (int*)calloc(*newpeak, sizeof(int));
}

for (int i = 0; i < peak; i++) {
    if (i == deli) continue;
    int new_i = (i < deli) ? i : i - 1;
    for (int j = 0; j < peak; j++) {
        if (j == deli) continue;
        int new_j = (j < deli) ? j : j - 1;
        if (i == keepi || j == keepi) {
            if (i == keepi && j == keepi) {
                buffer[new_i][new_j] = 0;
            }
            else if (i == keepi) {
                buffer[new_i][new_j] = (matrix[keepi][j] ||
matrix[deli][j]) ? 1 : 0;
            }
            else if (j == keepi) {
                buffer[new_i][new_j] = (matrix[i][keepi] ||
matrix[i][deli]) ? 1 : 0;
            }
        }
        else {

```

```

        buffer[new_i][new_j] = matrix[i][j];
    }
}
return buffer;
}

int** mergeMatrix(int** matrix, int peak, int p1, int p2, int*
newpeak) {
    if (p1 < 1 || p1 > peak || p2 < 1 || p2 > peak || p1 == p2)
    {
        printf("Ошибка: некорректные номера вершин!\n");
        *newpeak = peak;
        return copyMatrixAdj(matrix, peak);
    }
    int i1 = p1 - 1;
    int i2 = p2 - 1;
    if (matrix[i1][i2] == 0) {
        printf("Ошибка: ребро между вершинами %d и %d не
существует!\n", p1, p2);
        *newpeak = peak;
        return copyMatrixAdj(matrix, peak);
    }
    return identifiypeakMatrix(matrix, peak, p1, p2, newpeak);
}

int** splitMatrix(int** matrix, int peak, int p, int* newpeak) {
    if (p < 1 || p > peak) {
        printf("Ошибка: некорректный номер вершины!\n");
        *newpeak = peak;
        return copyMatrixAdj(matrix, peak);
    }
    int i = p - 1;

```

```

    *newpeak = peak + 1;
    int** buffer = (int**)malloc(*newpeak * sizeof(int*));
    for (int i = 0; i < *newpeak; i++) {
        buffer[i] = (int*)calloc(*newpeak, sizeof(int));
    }
    for (int i = 0; i < peak; i++) {
        for (int j = 0; j < peak; j++) {
            buffer[i][j] = matrix[i][j];
        }
    }
    int newpi = peak;
    buffer[i][newpi] = 1;
    buffer[newpi][i] = 1;
    for (int i = 0; i < *newpeak; i++) {
        buffer[i][i] = 0;
    }
    return buffer;
}

void freemMatrix(int** matrix, int peak) {
    for (int i = 0; i < peak; i++) {
        free(matrix[i]);
    }
    free(matrix);
}

Node* createNode(int v) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->vertex = v;
    newNode->next = NULL;
    return newNode;
}

```

```

Graph* createGraph(int peak) {
    Graph* graph = (Graph*)malloc(sizeof(Graph));
    graph->peak = peak;
    graph->adjLists = (Node**)malloc(peak * sizeof(Node*));

    for (int i = 0; i < peak; i++) {
        graph->adjLists[i] = NULL;
    }

    return graph;
}

void addEdge(Graph* graph, int src, int dest) {
    Node* newNode = createNode(dest);
    newNode->next = graph->adjLists[src];
    graph->adjLists[src] = newNode;

    newNode = createNode(src);
    newNode->next = graph->adjLists[dest];
    graph->adjLists[dest] = newNode;
}

void createMatrixList(Graph* graph, int peak) {
    for (int i = 0; i < peak; i++) {
        for (int j = i + 1; j < peak; j++) {
            if (rand() % 2 == 1) {
                addEdge(graph, i, j);
            }
        }
    }
}

```

```

void printMatrixList(Graph* graph) {
    printf("Списки смежности (%d вершин):\n", graph->peak);
    for (int i = 0; i < graph->peak; i++) {
        printf("Вершина %d: ", i + 1);
        Node* temp = graph->adjLists[i];
        if (temp == NULL) {
            printf("нет соседей");
        }
        else {
            while (temp) {
                printf("%d ", temp->vertex + 1);
                temp = temp->next;
            }
        }
        printf("\n");
    }
    printf("\n");
}

```

```

Graph* copyMatrixList(Graph* graph) {
    Graph* copy = createGraph(graph->peak);

    for (int i = 0; i < graph->peak; i++) {
        Node* temp = graph->adjLists[i];
        Node** copyPtr = &(copy->adjLists[i]);

        while (temp) {
            *copyPtr = createNode(temp->vertex);
            copyPtr = &((*copyPtr)->next);
            temp = temp->next;
        }
    }
}

```

```

    }

    return copy;
}

void removeVertexFromList(Node** head, int v) {
    Node* temp = *head;
    Node* prev = NULL;

    while (temp != NULL && temp->vertex != v) {
        prev = temp;
        temp = temp->next;
    }

    if (temp != NULL) {
        if (prev == NULL) {
            *head = temp->next;
        }
        else {
            prev->next = temp->next;
        }
        free(temp);
    }
}

int isVertexInList(Node* head, int v) {
    Node* temp = head;
    while (temp != NULL) {
        if (temp->vertex == v) {
            return 1;
        }
        temp = temp->next;
    }
}

```

```

    }
    return 0;
}

void freemList(Graph* graph) {
    for (int i = 0; i < graph->peak; i++) {
        Node* temp = graph->adjLists[i];
        while (temp) {
            Node* toDelete = temp;
            temp = temp->next;
            free(toDelete);
        }
    }
    free(graph->adjLists);
    free(graph);
}

Graph* identifiypeakList(Graph* graph, int p1, int p2, int*
newpeak) {
    if (p1 < 1 || p1 > graph->peak || p2 < 1 || p2 > graph->peak
|| p1 == p2) {
        printf("Ошибка: некорректные номера вершин!\n");
        *newpeak = graph->peak;
        return copyMatrixList(graph);
    }

    int i1 = p1 - 1;
    int i2 = p2 - 1;
    int keepi = (i1 < i2) ? i1 : i2;
    int deli = (i1 < i2) ? i2 : i1;

    *newpeak = graph->peak - 1;
    Graph* newGraph = createGraph(*newpeak);

```

```

Node* combinedNeighbors = NULL;

Node* temp = graph->adjLists[keepi];
while (temp != NULL) {
    if (temp->vertex != keepi && temp->vertex != deli) {
        Node* newNode = createNode(temp->vertex);
        newNode->next = combinedNeighbors;
        combinedNeighbors = newNode;
    }
    temp = temp->next;
}

temp = graph->adjLists[deli];
while (temp != NULL) {
    if (temp->vertex != keepi && temp->vertex != deli) {
        if (!isVertexInList(combinedNeighbors, temp-
>vertex)) {
            Node* newNode = createNode(temp->vertex);
            newNode->next = combinedNeighbors;
            combinedNeighbors = newNode;
        }
    }
    temp = temp->next;
}

for (int i = 0; i < graph->peak; i++) {
    if (i == deli) continue;
    int new_i = (i < deli) ? i : i - 1;
    Node* currentList = NULL;

    if (i == keepi) {

```



```

        currentList = combinedNeighbors;
    }
    else {
        temp = graph->adjLists[i];
        while (temp != NULL) {
            int old_j = temp->vertex;
            int new_j;

            if (old_j == deli) {
                new_j = keepi;
            }
            else {
                new_j = (old_j < deli) ? old_j : old_j - 1;
            }

            if (new_j != new_i &&
!isVertexInList(currentList, new_j)) {
                Node* newNode = createNode(new_j);
                newNode->next = currentList;
                currentList = newNode;
            }
            temp = temp->next;
        }
        newGraph->adjLists[new_i] = currentList;
    }

    return newGraph;
}

Graph* mergeList(Graph* graph, int p1, int p2, int* newpeak) {
    if (p1 < 1 || p1 > graph->peak || p2 < 1 || p2 > graph->peak
|| p1 == p2) {

```

```

        printf("Ошибка: некорректные номера вершин!\n");
        *newpeak = graph->peak;
        return copyMatrixList(graph);
    }

    int i1 = p1 - 1;
    int i2 = p2 - 1;

    if (!isVertexInList(graph->adjLists[i1], i2)) {
        printf("Ошибка: ребро между вершинами %d и %d не существует!\n", p1, p2);
        *newpeak = graph->peak;
        return copyMatrixList(graph);
    }

    Graph* result = identifiypeakList(graph, p1, p2, newpeak);
    int keepi = (i1 < i2) ? i1 : i1 - 1;
    removeVertexFromList(&(result->adjLists[keepi]), keepi);

    return result;
}

Graph* splitList(Graph* graph, int p, int* newpeak) {
    if (p < 1 || p > graph->peak) {
        printf("Ошибка: некорректный номер вершины!\n");
        *newpeak = graph->peak;
        return copyMatrixList(graph);
    }

    int i = p - 1;
    *newpeak = graph->peak + 1;
    Graph* newGraph = createGraph(*newpeak);

```

```

int newpi = graph->peak;

for (int i = 0; i < graph->peak; i++) {
    Node* temp = graph->adjLists[i];
    while (temp != NULL) {
        int old_j = temp->vertex;
        if (!isVertexInList(newGraph->adjLists[i], old_j) &&
i != old_j) {
            Node* newNode = createNode(old_j);
            newNode->next = newGraph->adjLists[i];
            newGraph->adjLists[i] = newNode;
        }
        temp = temp->next;
    }
}

if (!isVertexInList(newGraph->adjLists[i], newpi)) {
    Node* newNode1 = createNode(newpi);
    newNode1->next = newGraph->adjLists[i];
    newGraph->adjLists[i] = newNode1;

    Node* newNode2 = createNode(i);
    newNode2->next = newGraph->adjLists[newpi];
    newGraph->adjLists[newpi] = newNode2;
}

return newGraph;
}

int main() {
    setlocale(LC_ALL, "ru");
    srand(time(NULL));

```

```

int representationType;
printf("Выберите представление графа:\n");
printf("1 - Матрица смежности\n");
printf("2 - Списки смежности\n");
printf("Выбор: ");
scanf("%d", &representationType);

if (representationType != 1 && representationType != 2) {
    printf("Ошибка: некорректный выбор!\n");
    return 1;
}

int peak;
printf("Введите количество вершин для графа: ");
scanf("%d", &peak);

if (peak <= 0) {
    printf("Ошибка: количество вершин должно быть  
положительным числом.\n");
    return 1;
}

if (representationType == 1) {
    int** matrix = (int**)malloc(peak * sizeof(int*));
    for (int i = 0; i < peak; i++) {
        matrix[i] = (int*)malloc(peak * sizeof(int));
    }

    createMatrixAdj(matrix, peak);
    printf("\nИсходный граф\n");
    printMatrixAdj(matrix, peak);
}

```

```

int choice;
int p1, p2, p;
int newpeak;
do {
    printf("\nОперации над графом \n");
    printf("1 - Отождествление вершин\n");
    printf("2 - Стягивание ребра\n");
    printf("3 - Расщепление вершины\n");
    printf("0 - Выход\n");
    printf("Выберите операцию: ");
    scanf("%d", &choice);
    switch (choice) {
        case 1: {
            printf("Введите номера вершин для отождествления\n(через пробел): ");
            scanf("%d %d", &p1, &p2);
            int** result = identifiypeakMatrix(matrix, peak,
p1, p2, &newpeak);
            printf("\nРезультат отождествления вершин %d и\n%d:\n", p1, p2);
            printMatrixAdj(result, newpeak);
            freemMatrix(result, newpeak);
            break;
        }
        case 2: {
            printf("Введите номера вершин ребра для\nстягивания (через пробел): ");
            scanf("%d %d", &p1, &p2);
            int** result = mergeMatrix(matrix, peak, p1, p2,
&newpeak);
            printf("\nРезультат стягивания ребра (%d,\nd):\n", p1, p2);
            printMatrixAdj(result, newpeak);

```

```

        freemMatrix(result, newpeak);
        break;
    }
    case 3: {
        printf("Введите номер вершины для расщепления:
");
        scanf("%d", &p);
        int** result = splitMatrix(matrix, peak, p,
&newpeak);
        printf("\nРезультат расщепления вершины %d:\n",
p);
        printMatrixAdj(result, newpeak);
        freemMatrix(result, newpeak);
        break;
    }
    case 0:
        printf("Выход из программы.\n");
        break;
    default:
        printf("Неверный выбор!\n");
    }
} while (choice != 0);
freemMatrix(matrix, peak);
}

else if (representationType == 2) {
    Graph* graph = createGraph(peak);
    createMatrixList(graph, peak);
    printf("\nИсходный граф\n");
    printMatrixList(graph);

    int choice;
    int p1, p2, p;
    int newpeak;

```

```

do {

    printf("\nОперации над графом\n");
    printf("1 - Отождествление вершин\n");
    printf("2 - Стыгивание ребра\n");
    printf("3 - Расщепление вершины\n");
    printf("0 - Выход\n");
    printf("Выберите операцию: ");
    scanf("%d", &choice);

    Graph* result = NULL;

    switch (choice) {
    case 1: {
        printf("Введите номера вершин для отождествления
(через пробел): ");
        scanf("%d %d", &p1, &p2);
        result = identifiypeakList(graph, p1, p2,
&newpeak);
        printf("\nРезультат отождествления вершин %d и
%d:\n", p1, p2);
        printMatrixList(result);
        freemList(result);
        break;
    }
    case 2: {
        printf("Введите номера вершин ребра для
стыгивания (через пробел): ");
        scanf("%d %d", &p1, &p2);
        result = mergeList(graph, p1, p2, &newpeak);
        printf("\nРезультат стыгивания ребра (%d,
%d):\n", p1, p2);
        printMatrixList(result);
        freemList(result);
    }
    }
}

```

```

        break;
    }
    case 3: {
        printf("Введите номер вершины для расщепления:
");
        scanf("%d", &p);
        result = splitList(graph, p, &newpeak);
        printf("\nРезультат расщепления вершины %d:\n",
p);

        printMatrixList(result);
        freemList(result);
        break;
    }
    case 0:
        printf("Выход из программы.\n");
        break;
    default:
        printf("Неверный выбор!\n");
    }
} while (choice != 0);

freemList(graph);
}

return 0;
}

```


3-ие задание

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>

void createMatrix(int** matrix, int peak) {
    for (int i = 0; i < peak; i++) {
        for (int j = i + 1; j < peak; j++) {
            int value = rand() % 2;
            matrix[i][j] = value;
            matrix[j][i] = value;
        }
        matrix[i][i] = 0;
    }
}

void printMatrix(int** matrix, int peak, const char* name) {
    printf("%s (%dx%d):\n", name, peak, peak);
    printf("    ");
    for (int j = 0; j < peak; j++) {
        printf("%2d ", j + 1);
    }
    printf("\n");

    for (int i = 0; i < peak; i++) {
        printf("%2d ", i + 1);
        for (int j = 0; j < peak; j++) {
            printf("%2d ", matrix[i][j]);
        }
        printf("\n");
    }
}
```

```
    }  
    printf("\n");  
}
```

```
int** createMatrixFunc(int peak) {  
    int** matrix = (int**)malloc(peak * sizeof(int*));  
    for (int i = 0; i < peak; i++) {  
        matrix[i] = (int*)calloc(peak, sizeof(int));  
    }  
    return matrix;  
}
```

```
void freem(int** matrix, int peak) {  
    for (int i = 0; i < peak; i++) {  
        free(matrix[i]);  
    }  
    free(matrix);  
}
```

```
int** copyMatrix(int** source, int peak) {  
    int** copy = createMatrixFunc(peak);  
    for (int i = 0; i < peak; i++) {  
        for (int j = 0; j < peak; j++) {  
            copy[i][j] = source[i][j];  
        }  
    }  
    return copy;  
}
```

```
int** unionGraphs(int** matrix1, int** matrix2, int peak) {  
    int** result = createMatrixFunc(peak);  
    for (int i = 0; i < peak; i++) {
```

```

        for (int j = 0; j < peak; j++) {
            result[i][j] = (matrix1[i][j] || matrix2[i][j]) ? 1
: 0;
        }
    }
    return result;
}

int** intersectionGraphs(int** matrix1, int** matrix2, int peak)
{
    int** result = createMatrixFunc(peak);
    for (int i = 0; i < peak; i++) {
        for (int j = 0; j < peak; j++) {
            result[i][j] = (matrix1[i][j] && matrix2[i][j]) ? 1
: 0;
        }
    }
    return result;
}

int** ringSumGraphs(int** matrix1, int** matrix2, int peak) {
    int** result = createMatrixFunc(peak);
    for (int i = 0; i < peak; i++) {
        for (int j = 0; j < peak; j++) {
            result[i][j] = (matrix1[i][j] != matrix2[i][j]) ? 1
: 0;
        }
    }
    return result;
}

int main() {
    setlocale(LC_ALL, "ru");

```

```

srand(time(NULL));

int peak;

printf("Введите количество вершин для графов G1 и G2: ");
scanf("%d", &peak);

if (peak <= 0) {
    printf("Ошибка: количество вершин должно быть
положительным числом.\n");
    return 1;
}

int** matrix1 = createMatrixFunc(peak);
int** matrix2 = createMatrixFunc(peak);

createMatrix(matrix1, peak);
createMatrix(matrix2, peak);

printf("\nИсходные графы\n");
printMatrix(matrix1, peak, "Матрица смежности G1");
printMatrix(matrix2, peak, "Матрица смежности G2");

int choice;
do {
    printf("\nОперации над графами\n");
    printf("1 - Объединение G1 U G2\n");
    printf("2 - Пересечение G1 n G2\n");
    printf("3 - Сложение по модулю (2) G1 (+) G2\n");
    printf("0 - Выход\n");
    printf("Выберите операцию: ");
    scanf("%d", &choice);

    int** result = NULL;

```

```

const char* operationName = "";

switch (choice) {
case 1:
    result = unionGraphs(matrix1, matrix2, peak);
    operationName = "Результат объединения  $G_1 \cup G_2$ ";
    break;
case 2:
    result = intersectionGraphs(matrix1, matrix2, peak);
    operationName = "Результат пересечения  $G_1 \cap G_2$ ";
    break;
case 3:
    result = ringSumGraphs(matrix1, matrix2, peak);
    operationName = "Результат сложения по модулю (2)  $G_1$ 
(+)  $G_2$ ";
    break;
case 0:
    printf("Выход из программы.\n");
    break;
default:
    printf("Неверный выбор!\n");
}

if (result != NULL) {
    printf("\n%s\n", operationName);
    printMatrix(result, peak, "");
    freem(result, peak);
}

} while (choice != 0);

freem(matrix1, peak);
freem(matrix2, peak);

```

```
        return 0;
    }
}
```

4-ое задание

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>

void createMatrix(int** matrix, int peak) {
    for (int i = 0; i < peak; i++) {
        for (int j = i + 1; j < peak; j++) {
            int value = rand() % 2;
            matrix[i][j] = value;
            matrix[j][i] = value;
        }
        matrix[i][i] = 0;
    }
}

void printMatrix(int** matrix, int peak, const char* name) {
    printf("%s (%dx%d):\n", name, peak, peak);
    printf("    ");
    for (int j = 0; j < peak; j++) {
        printf("%2d ", j + 1);
    }
    printf("\n");

    for (int i = 0; i < peak; i++) {
        printf("%2d ", i + 1);
        for (int j = 0; j < peak; j++) {
            printf("%2d ", matrix[i][j]);
        }
    }
}
```

```

        }
        printf("\n");
    }
    printf("\n");
}

int** createMatrixFunc(int peak) {
    int** matrix = (int**)malloc(peak * sizeof(int*));
    for (int i = 0; i < peak; i++) {
        matrix[i] = (int*)calloc(peak, sizeof(int));
    }
    return matrix;
}

void freem(int** matrix, int peak) {
    for (int i = 0; i < peak; i++) {
        free(matrix[i]);
    }
    free(matrix);
}

int** cartesianProduct(int** matrix1, int peak1, int** matrix2,
int peak2, int* resultPeak) {
    *resultPeak = peak1 * peak2;
    int** result = createMatrixFunc(*resultPeak);

    for (int u1 = 0; u1 < peak1; u1++) {
        for (int v1 = 0; v1 < peak2; v1++) {
            for (int u2 = 0; u2 < peak1; u2++) {
                for (int v2 = 0; v2 < peak2; v2++) {
                    int i = u1 * peak2 + v1;
                    int j = u2 * peak2 + v2;

```

```

        if ((u1 == u2 && matrix2[v1][v2] == 1) ||
(v1 == v2 && matrix1[u1][u2] == 1)) {
            result[i][j] = 1;
        }
    }
}
}
}
return result;
}

```

```

void printCartesianMatrix(int** matrix, int peak1, int peak2,
const char* name) {

```

```

    int totalPeak = peak1 * peak2;
    printf("%s (%dx%d):\n", name, totalPeak, totalPeak);
    printf("    ");
    for (int u = 0; u < peak1; u++) {
        for (int v = 0; v < peak2; v++) {
            printf("(%d,%d) ", u + 1, v + 1);
        }
    }
    printf("\n");

```

```

    for (int u1 = 0; u1 < peak1; u1++) {
        for (int v1 = 0; v1 < peak2; v1++) {
            int i = u1 * peak2 + v1;
            printf("(%d,%d)", u1 + 1, v1 + 1);
            for (int u2 = 0; u2 < peak1; u2++) {
                for (int v2 = 0; v2 < peak2; v2++) {
                    int j = u2 * peak2 + v2;
                    printf("%2d    ", matrix[i][j]);
                }
            }
        }
    }
}

```



```

        printf("\n");
    }
}

printf("\n");
}

void printCartesianAdjacencyList(int** matrix, int peak1, int
peak2) {
    int totalPeak = peak1 * peak2;
    printf("Списки смежности декартова произведения:\n");
    for (int u1 = 0; u1 < peak1; u1++) {
        for (int v1 = 0; v1 < peak2; v1++) {
            int i = u1 * peak2 + v1;
            printf("Вершина (%d,%d): ", u1 + 1, v1 + 1);
            int hasNeighbors = 0;
            for (int u2 = 0; u2 < peak1; u2++) {
                for (int v2 = 0; v2 < peak2; v2++) {
                    int j = u2 * peak2 + v2;
                    if (matrix[i][j] == 1 && i != j) {
                        printf("(%d,%d) ", u2 + 1, v2 + 1);
                        hasNeighbors = 1;
                    }
                }
            }
            if (!hasNeighbors) {
                printf("нет соседей");
            }
            printf("\n");
        }
    }
    printf("\n");
}

```

```

int main() {
    setlocale(LC_ALL, "ru");
    srand(time(NULL));
    int peak1, peak2;
    printf("Введите количество вершин для графа G1: ");
    scanf("%d", &peak1);
    printf("Введите количество вершин для графа G2: ");
    scanf("%d", &peak2);

    if (peak1 <= 0 || peak2 <= 0) {
        printf("Ошибка: количество вершин должно быть
положительным числом.\n");
        return 1;
    }

    int** matrix1 = createMatrixFunc(peak1);
    int** matrix2 = createMatrixFunc(peak2);

    createMatrix(matrix1, peak1);
    createMatrix(matrix2, peak2);

    printf("\nИсходные графы\n");
    printMatrix(matrix1, peak1, "Матрица смежности G1");
    printMatrix(matrix2, peak2, "Матрица смежности G2");

    int resultPeak;

    int** result = cartesianProduct(matrix1, peak1, matrix2,
peak2, &resultPeak);

    printf("\nДекартово произведение G1 X G2\n");

    printf("Количество вершин в результате: %d x %d = %d\n\n",
peak1, peak2, resultPeak);

```

```
    printCartesianMatrix(result, peak1, peak2, "Матрица  
смежности декартова произведения");
```

```
    printCartesianAdjacencyList(result, peak1, peak2);
```

```
    freeem(matrix1, peak1);
```

```
    freeem(matrix2, peak2);
```

```
    freeem(result, resultPeak);
```

```
    return 0;
```

```
}
```

Результат работы программы

1-ое:

```
Консоль отладки Microsoft Visual Studio
Введите количество вершин для графа G1: 5
Введите количество вершин для графа G2: 5

Граф G1
Матрица смежности:
0 1 0 1 0
1 0 0 1 0
0 0 0 1 1
1 1 1 0 0
0 0 1 0 0

Список смежности:
Вершина 1: 2 4
Вершина 2: 1 4
Вершина 3: 4 5
Вершина 4: 1 2 3
Вершина 5: 3

Граф G2
Матрица смежности:
0 1 1 1 1
1 0 0 1 1
1 0 0 1 0
1 1 1 0 1
1 1 0 1 0

Список смежности:
Вершина 1: 2 3 4 5
Вершина 2: 1 4 5
Вершина 3: 1 4
Вершина 4: 1 2 3 5
Вершина 5: 1 2 4
```

2-ое:

C:\Users\amaka\source\repos\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe

Выберите представление графа:

1 - Матрица смежности

2 - Списки смежности

Выбор: 1

Введите количество вершин для графа: 3

Исходный граф

Матрица смежности (3x3):

	1	2	3
1	0	1	0
2	1	0	1
3	0	1	0

Операции над графом

1 - Отождествление вершин

2 - Стягивание ребра

3 - Расщепление вершины

0 - Выход

Выберите операцию: 1

Введите номера вершин для отождествления (через пробел): 1 3

Результат отождествления вершин 1 и 3:

Матрица смежности (2x2):

	1	2
1	0	1
2	1	0

Операции над графом

1 - Отождествление вершин

2 - Стягивание ребра

3 - Расщепление вершины

0 - Выход

Выберите операцию: 3

Введите номер вершины для расщепления: 1

Результат расщепления вершины 1:

Матрица смежности (4x4):

	1	2	3	4
1	0	1	0	1
2	1	0	1	0
3	0	1	0	0
4	1	0	0	0

Операции над графом

1 - Отождествление вершин

2 - Стягивание ребра

3 - Расщепление вершины

0 - Выход

```
C:\Users\amaka\source\repos\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe
Выберите представление графа:
1 - Матрица смежности
2 - Списки смежности
Выбор: 2
Введите количество вершин для графа: 3

Исходный граф
Списки смежности (3 вершин):
Вершина 1: 3 2
Вершина 2: 3 1
Вершина 3: 2 1

Операции над графом
1 - Отождествление вершин
2 - Стягивание ребра
3 - Расщепление вершины
0 - Выход
Выберите операцию: 1
Введите номера вершин для отождествления (через пробел): 1 3

Результат отождествления вершин 1 и 3:
Списки смежности (2 вершин):
Вершина 1: 2
Вершина 2: 1

Операции над графом
1 - Отождествление вершин
2 - Стягивание ребра
3 - Расщепление вершины
0 - Выход
Выберите операцию: 3
Введите номер вершины для расщепления: 1

Результат расщепления вершины 1:
Списки смежности (4 вершин):
Вершина 1: 4 2 3
Вершина 2: 1 3
Вершина 3: 1 2
Вершина 4: 1

Операции над графом
1 - Отождествление вершин
2 - Стягивание ребра
3 - Расщепление вершины
0 - Выход
```

3-ие:

C:\Users\amaka\source\repos\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe

Введите количество вершин для графов G1 и G2: 4

Исходные графы

Матрица смежности G1 (4x4):

	1	2	3	4
1	0	1	0	0
2	1	0	1	0
3	0	1	0	1
4	0	0	1	0

Матрица смежности G2 (4x4):

	1	2	3	4
1	0	0	0	1
2	0	0	0	1
3	0	0	0	0
4	1	1	0	0

Операции над графами

1 - Объединение $G1 \cup G2$

2 - Пересечение $G1 \cap G2$

3 - Сложение по модулю (2) $G1 (+) G2$

0 - Выход

Выберите операцию: 1

Результат объединения $G1 \cup G2$

(4x4):

	1	2	3	4
1	0	1	0	1
2	1	0	1	1
3	0	1	0	1
4	1	1	1	0

```
C:\Users\amaka\source\repos\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe
Введите количество вершин для графов G1 и G2: 3

Исходные графы
Матрица смежности G1 (3x3):
  1  2  3
1  0  1  0
2  1  0  1
3  0  1  0

Матрица смежности G2 (3x3):
  1  2  3
1  0  1  0
2  1  0  1
3  0  1  0

Операции над графами
1 - Объединение G1 U G2
2 - Пересечение G1 n G2
3 - Сложение по модулю (2) G1 (+) G2
0 - Выход
Выберите операцию: 2

Результат пересечения G1 n G2
(3x3):
  1  2  3
1  0  1  0
2  1  0  1
3  0  1  0
```

4-ое:


```

Консоль отладки Microsoft Visual Studio
Введите количество вершин для графа G1: 3
Введите количество вершин для графа G2: 3

Исходные графы
Матрица смежности G1 (3x3):
    1  2  3
1  0  1  0
2  1  0  1
3  0  1  0

Матрица смежности G2 (3x3):
    1  2  3
1  0  1  1
2  1  0  0
3  1  0  0

Декартово произведение G1 X G2
Количество вершин в результате: 3 x 3 = 9

Матрица смежности декартова произведения (9x9):
    (1,1) (1,2) (1,3) (2,1) (2,2) (2,3) (3,1) (3,2) (3,3)
(1,1) 0     1     1     1     0     0     0     0     0
(1,2) 1     0     0     0     1     0     0     0     0
(1,3) 1     0     0     0     0     1     0     0     0
(2,1) 1     0     0     0     1     1     1     0     0
(2,2) 0     1     0     1     0     0     0     1     0
(2,3) 0     0     1     1     0     0     0     0     1
(3,1) 0     0     0     1     0     0     0     1     1
(3,2) 0     0     0     0     1     0     1     0     0
(3,3) 0     0     0     0     0     1     1     0     0

Списки смежности декартова произведения:
Вершина (1,1): (1,2) (1,3) (2,1)
Вершина (1,2): (1,1) (2,2)
Вершина (1,3): (1,1) (2,3)
Вершина (2,1): (1,1) (2,2) (2,3) (3,1)
Вершина (2,2): (1,2) (2,1) (3,2)
Вершина (2,3): (1,3) (2,1) (3,3)
Вершина (3,1): (2,1) (3,2) (3,3)
Вершина (3,2): (2,2) (3,1)
Вершина (3,3): (2,3) (3,1)

```

Вывод

В результате выполнения работы были изучены и применены унарные операции над графами, разделенные на две основные группы в зависимости от их влияния на структуру графа. Операции первой группы, включающие отождествление вершин и стягивание ребра, позволяют упрощать структуру графа путем уменьшения числа его элементов, что находит применение в задачах оптимизации и анализа сложных сетей. Операции второй группы,

такие как расщепление вершины и добавление ребра, используются для построения более сложных графовых структур и моделирования расширяющихся систем.